Informe de práctica de la semana 1

Diagnóstico y refactorización de un sistema académico

UNAP · Ingeniería de Sistemas · SIS206 POO II · 2026 II
S1-001 · 19 de octubre de 2026 · Informe revisado

1 Problema y línea base

Se desarrollaron las cinco actividades iniciales y la Parte II de la guía [1]. El sistema académico quedó ejecutable en Python y C++, con resultados válidos conservados y nuevas validaciones de notas y duplicados. Este informe explica el problema inicial, las responsabilidades elegidas y la evidencia que permite comprobar la refactorización.

El ejemplo de la primera parte devuelve 15.0 para las notas 16 y 14. El gestor de la Parte II genera E001;Ana;10.50;APROBADO para 10 y 11, pero también acepta 25. Se conservaron los originales, se ejecutaron en un entorno temporal y se registraron seis pruebas de caracterización antes de refactorizar.

2 Problemas de diseño identificados

Ubicación en el ejemplo inicial	Hallazgo	Corrección aplicada
Línea 2	Clase est poco expresiva	Estudiante como concepto del dominio
Líneas 3 a 23	Miembros y métodos abreviados	Nombres que explican su intención
Líneas 4 a 7	Estado público sin control	Propiedades de lectura y colección protegida
Líneas 9 y 10	Calificación sin validar	Rechazar valores fuera de 0 a 20
Líneas 10 y 17	Registros posicionales	Nota con curso y calificación
Líneas 20 a 23	Impresión dentro de la entidad	Consola en el cliente y generadores separados
Líneas 3 a 7	Constructor sin contrato mínimo	Nombre, código y escuela válidos
Líneas 2 a 23	Reglas y significado sin documentar	Contratos explícitos y pruebas

Las líneas se refieren a baseline/estudiante_original.py. El caso sin notas ya existía en el original. En la Parte II también se identificaron dependencia directa de archivos, código duplicado permitido y repetición potencial de reglas al agregar formatos.

3 Responsabilidades propuestas

El diseño separa las reglas académicas de los destinos de salida. ServicioAcademico recibe repositorio y auditoría; el generador se elige al solicitar el reporte. Las dependencias quedan visibles y pueden sustituirse para realizar pruebas sin archivos.

Componente	Responsabilidad	Límite
Nota y Estudiante	Datos válidos, promedio y aprobación	Sin archivos, consola ni SQL
RepositorioEstudiantes	Guardar y buscar por código	Sin reglas de formato de reporte
GeneradorCSV y GeneradorJSON	Convertir datos en texto	Sin persistir estudiantes
Auditoría	Registrar eventos en memoria o archivo	Sin calcular promedios
ServicioAcademico	Coordinar registro, notas y reportes	Recibe sus dependencias

4 Refactorizaciones aplicadas

En Python, Nota es una dataclass inmutable. Estudiante usa atributos internos y propiedades de lectura; la consulta de notas devuelve una tupla. agregar_nota valida antes de insertar y calcular_ppa devuelve 0.0 sin notas. El promedio es aritmético porque la guía no proporciona créditos para ponderarlo. __str__ devuelve una descripción y el cliente imprime.

En C++, Estudiante conserva miembros privados y getters const. agregarNota rechaza valores no finitos o fuera del intervalo mediante std::invalid_argument. Las demostraciones generan la misma salida que Python: PPA 15.00 y error al intentar registrar 25. Se verificaron C++17 y C++20 para cubrir las dos versiones solicitadas en la guía.

Se declararon reglas mínimas para nombre, escuela, curso y código. El código se normaliza a texto para admitir el ejemplo numérico y E001. Las nuevas validaciones cambian deliberadamente la conducta ante entradas inválidas; los resultados válidos se conservaron y se compararon con la base.

Análisis comparativo de la actividad 4

El diseño B tiene mayor cohesión porque cada clase atiende una finalidad. En A, cambiar el formato obliga a editar la misma clase que administra notas. Esa mezcla incumple responsabilidad única. B permite probar promedios y reportes de manera aislada, con auditoría en memoria. El costo es tener más componentes y preparar su ensamblaje.

En Python, el prefijo _ es una convención y @property controla la interfaz habitual; no ofrece la restricción de private de C++. El sufijo _ de un miembro C++ tampoco determina su privacidad: la define el control de acceso.

5 Resultados de pruebas y métricas

Entorno ejecutado: Python 3.12.14 y g++ 10.3.0 en Windows 11. Se aprobaron 30 pruebas Python y 24 pruebas C++ por estándar. Todas las compilaciones finales usaron -Wall -Wextra -Wpedantic -Werror. La salida principal y seis casos CSV coinciden entre lenguajes.

Caso comprobado	Resultado esperado y obtenido
Sin notas	Promedio 0 y estado desaprobado
16 y 14; tres notas 12, 14 y 16	Promedios 15 y 14, respectivamente
Extremos 0 y 20; umbral 10 y 11	Notas aceptadas; aprobación en promedio 10.5
-0.1, 20.1, 25 y valores no finitos	Rechazo sin agregar la nota
Registro, búsqueda y código repetido	Recuperación correcta y rechazo del duplicado
CSV, JSON y auditoría	Reportes sin cambiar notas; eventos válidos registrados

EXP POO 001 agregó JSON manteniendo CSV. En la variante monolítica se editó la clase existente; en la separada se agregó GeneradorJSON. Los recuentos siguientes corresponden a Python y excluyen pruebas, demostraciones y documentación del impacto productivo.

Indicador	Antes	Después
Archivos productivos antes de agregar JSON	1	5
Clases antes de agregar JSON	1	7
Métodos funcionales de Gestor o Servicio	3	3
Dependencias de archivos o consola en la entidad	2	0
Archivos afectados al agregar JSON	1	1
Clases existentes editadas al agregar JSON	1	0
Clases nuevas al agregar JSON	0	1
Regresiones en los seis casos válidos	0	0
Tiempo de elaboración asistida en minutos	0.142	0.384

Criterio: se cuentan archivos editados o agregados y métodos públicos funcionales, excluyendo constructores. Los tiempos registran una ejecución asistida por IA por variante, en orden fijo y con distinta cantidad de pruebas finales. No equivalen a tiempo del estudiante ni permiten concluir una ventaja general de velocidad. La evidencia está en metricas.csv y evidencias/.

6 Proyecto integrador y discusión

Actividad 5. Se mantuvo el tema del proyecto local [2]: sistema de reservas de hotel en Puno. Atiende a recepción, administración y encargados de reservas; busca organizar huéspedes, habitaciones y solicitudes. Se propusieron tres clases con atributos privados, sin relaciones en esta etapa.

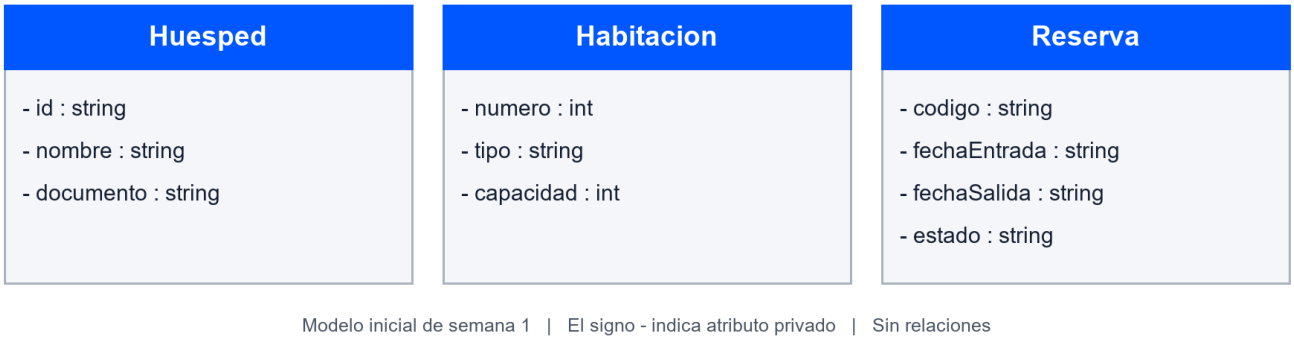


Figura 1 Modelo inicial del proyecto integrador. Fuente: propuesta de esta práctica basada en el tema del hotel. Se entrega PNG y archivo Draw.io editable.

La incorporación de JSON agregó un generador sin editar las clases existentes de la versión separada. La entidad quedó libre de archivos y consola, y los seis casos válidos conservaron su resultado. Aumentaron las clases y módulos porque se distribuyeron responsabilidades; la mejora se observa en el cambio localizado y en la posibilidad de probar cada regla.

Una limitación es la falta de transacciones: un fallo de auditoría posterior al registro no revierte la nota. Si el repositorio migrara a una base de datos, serían necesarias pruebas de integración y una política de atomicidad. El experimento de una sola sesión tampoco sustituye un estudio con participantes.

El historial local registra baseline, tests, refactor y final como etapas de preparación asistida. Antes de entregar, completar los datos del estudiante, renombrar el notebook y vincular GitHub. Las respuestas de análisis e investigación están en ACTIVIDADES_RESUELTAS.md.

7 Transparencia y entrega

Codex asistió en el análisis, desarrollo, ejecución, pruebas, diagramas y redacción. Las métricas describen esa preparación asistida. El estudiante debe estudiar las decisiones y realizar la sustentación y la revisión que solicite el curso.

8 Referencias

[1] Zanabria Gálvez, A. H. (2026). *Guía de práctica de laboratorio de la semana 1*. SIS206 POO II, Universidad Nacional del Altiplano. Material del curso.

[2] Material local del proyecto. (s. f.). *Plan de trabajo y README del sistema de reservas de hotel POO II*. Documentos proporcionados.

[3] Python Software Foundation. (s. f.). *abc — Abstract Base Classes*. Python 3.12 documentation. https://docs.python.org/3.12/library/abc.html

[4] Fowler, M. (2018). *Refactoring Improving the Design of Existing Code* (2.ª ed.). Addison Wesley. https://martinfowler.com/books/refactoring.html